Systèmes de gestion de bases de données

Reprise sur panne

P. Rigaux

Cnam, dépt. informatique

July 1, 2016

La notion de panne

Panne = tout dysfonctionnement affectant le comportement du système

Par souci de simplicité on va distinguer deux types de panne (quelle que soit la cause).

- **Panne légère** : affecte la RAM du serveur de données, pas les disques
- **Panne lourde** : affecte un disque

Dans un premier temps, on se concentre sur les pannes légères.

Garantie en cas de panne

Souvenons-nous des propriétés des transactions.

Durabilité et atomicité : quand le système rend la main après un `commit`, **toutes** les modifications de la transaction deviennent **permanentes**.

Recouvrabilité et Atomicité : tant qu'un `commit` n'a pas eu lieu, **toutes** les modifications de la transaction doivent pouvoir être **annulées** par un `rollback`.

Garantir un `commit`

Pour garantir le `commit`, la condition suivante doit être respectée.

Les données modifiées par une transaction validée doivent être sur le disque

On désigne ces données par le terme ”image après”.

Garantir un `rollback`

Pour garantir le `rollback`, la condition suivante doit être respectée.

Les données *avant* modification doivent être sur le disque

On désigne ces données par le terme ”image avant”.

L'état de la base

État de la base: état résultant de l'ensemble des transactions **validées** depuis l'origine.

On résume ce qui précède par:

L'état de la base doit *toujours* être sur le disque.

La difficulté

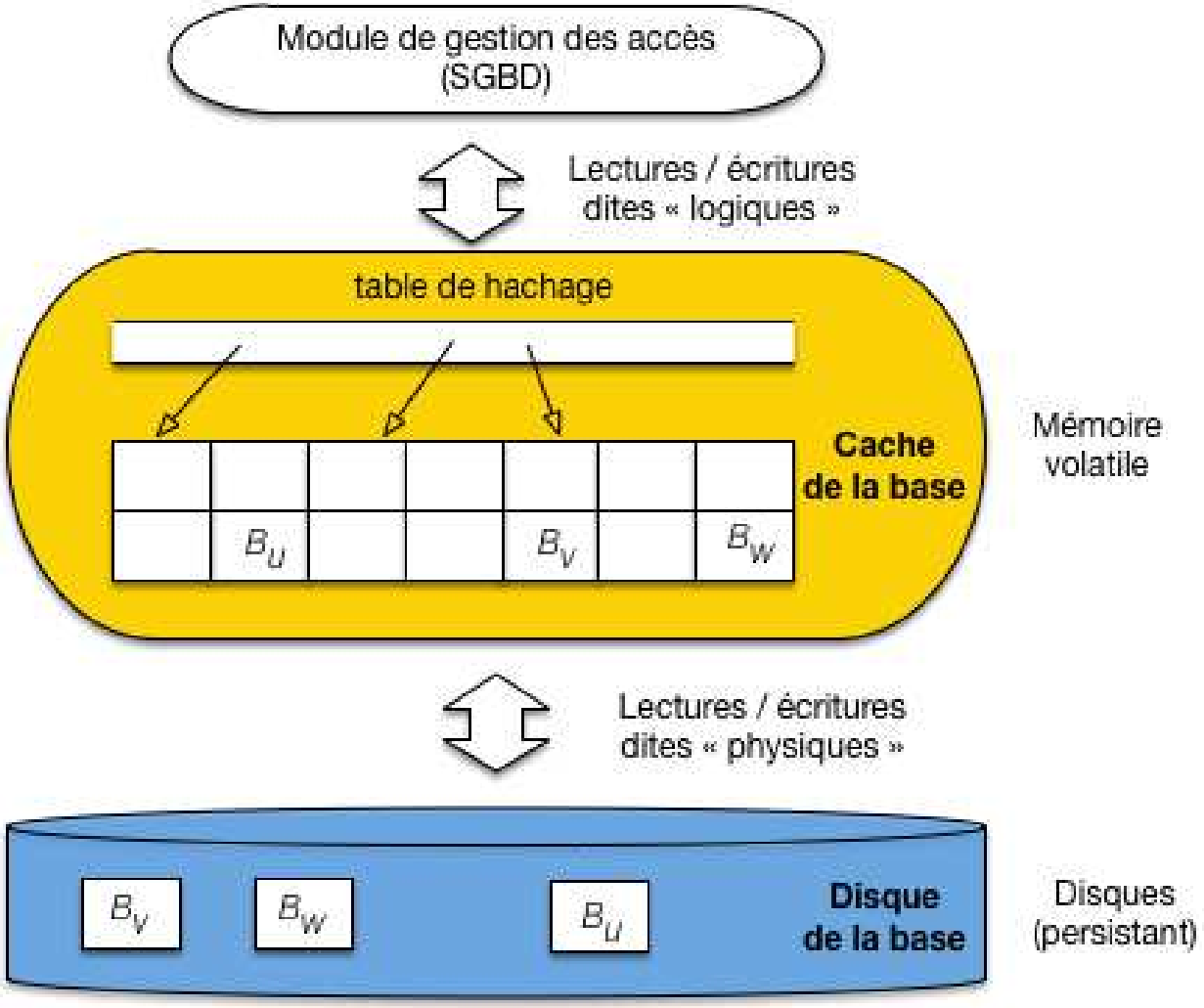
On peut résumer la difficulté ainsi :

- Jusqu'au `commit`, c'est **l'image avant** qui fait partie de l'état de la base.
- au `commit`, **l'image après** remplace **l'image avant** dans l'état de la base.

Problème : Comment assurer que ce remplacement s'effectue de manière **atomique** ("tout ou rien") ?

Ce n'est pas facile...

Le buffer et le disque



Opération de lecture

Toute opération de lecture contient l'adresse du bloc à lire.

Si le bloc est dans le buffer, le système accède à la donnée dans le bloc, et la retourne.

Sinon il faut d'abord lire un bloc du disque, et le placer dans le buffer.

Le SGBD garde en mémoire les blocs après lecture.

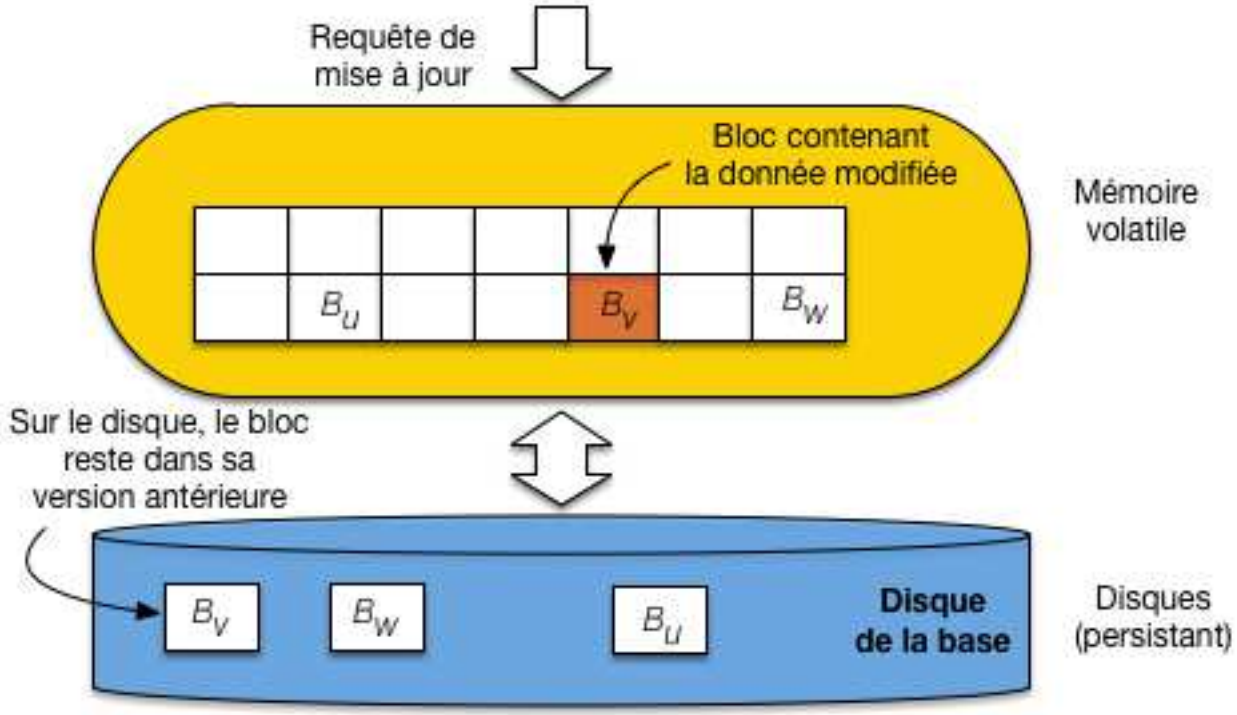
Stratégie de remplacement

Que faire quand la mémoire est pleine?

L'algorithme le plus courant est dit Least Recently Used (LRU).

- La "victime" est le bloc dont la dernière date de lecture logique est la plus ancienne.
- Ce bloc est alors soustrait de la mémoire centrale.
- Le nouveau bloc vient le remplacer.

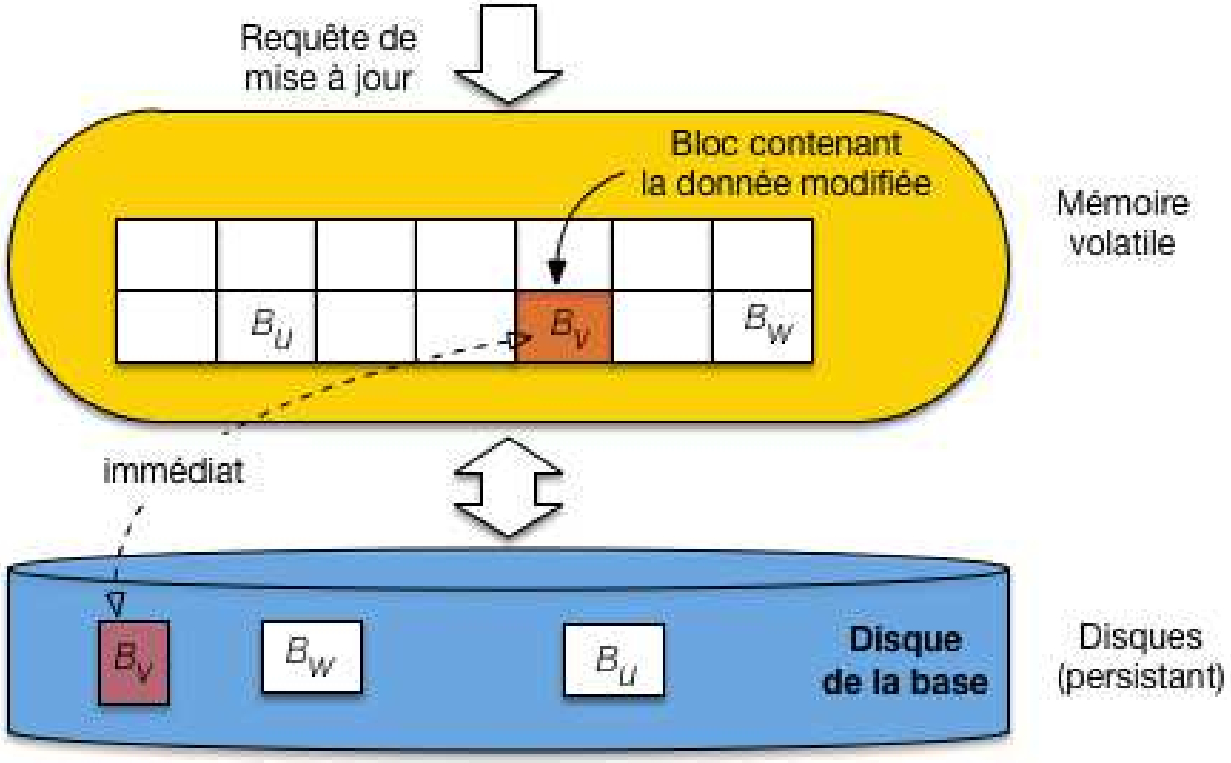
Opération de mise à jour



On trouve le bloc (comme pour une lecture), on modifie la donnée.

Faut-il écrire le bloc ou non?

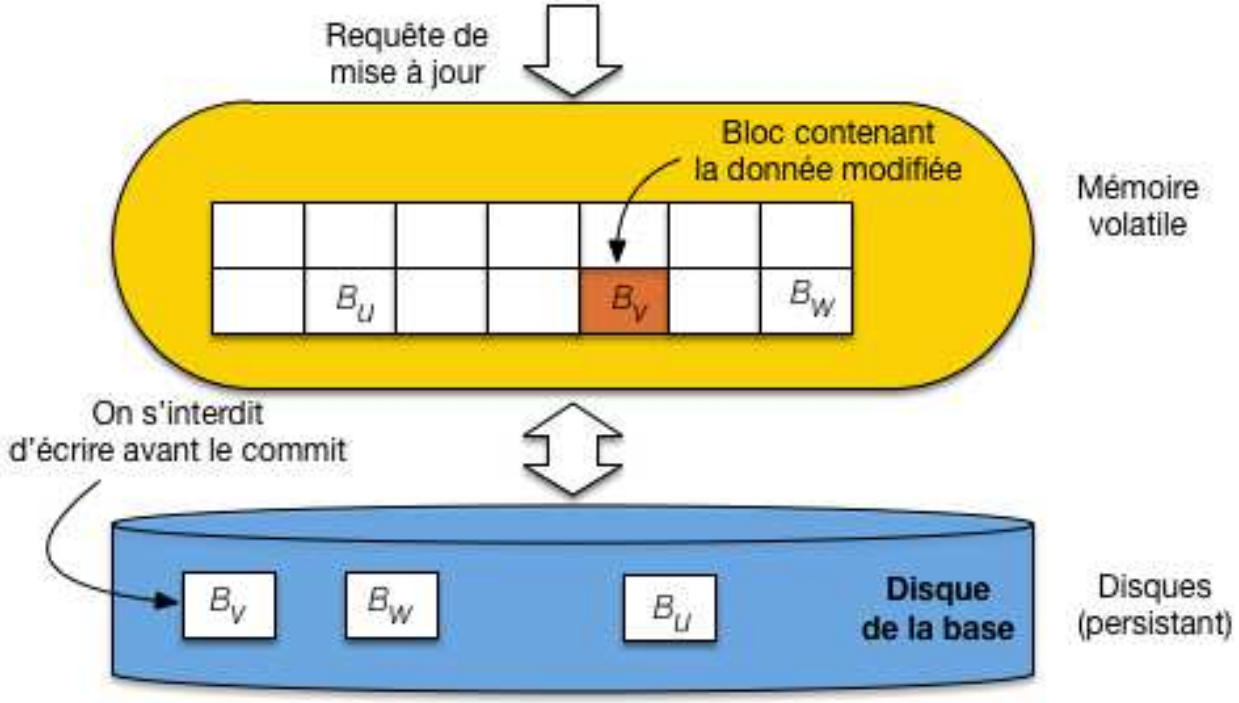
Mise à jour immédiate



Dès qu'un bloc est modifié, on écrit sur le disque pour synchroniser.

- Peu performant: **écritures aléatoires**
- **On écrase l'image avant**

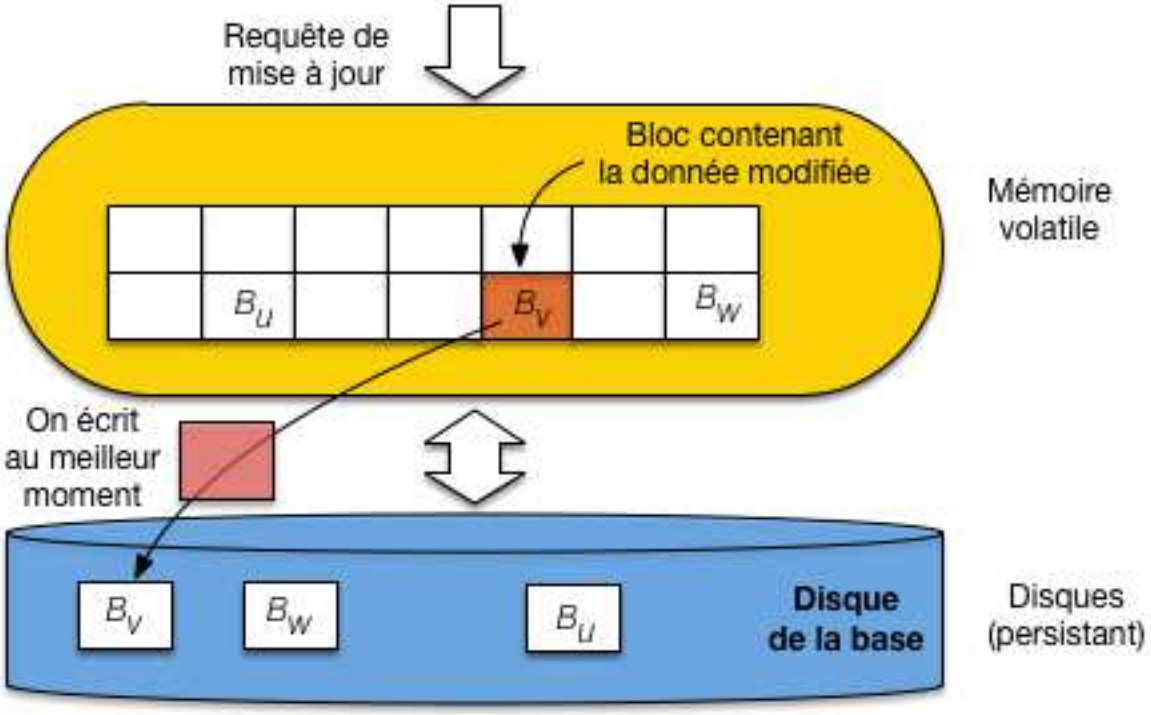
Mise à jour différée



On **interdit** l'écriture d'un bloc modifié avant le `commit`

- Risque de surcharger le buffer
- Préserve l'image avant

Mise à jour opportuniste



C'est la méthode la plus souple: on attend la meilleure opportunité pour synchroniser le buffer et le disque.

Le moins pénalisant; permet des écritures successives dans le buffer.

Résumé: lecture/écriture, buffer et disque

Retenir:

- Une partie de la base est copiée dans le buffer.
- Tout mise à jour s'effectue **d'abord dans le buffer**.
- La synchronisation avec le disque peut s'effectuer de manière **immédiate**, **différée**, ou **opportuniste**.

La méthode de synchronisation a un impact fort sur la reprise sur panne.

Comment reprendre sur panne légère

Une transaction s'exécute; **l'image avant** est sur le disque; **l'image après** est dans le buffer.

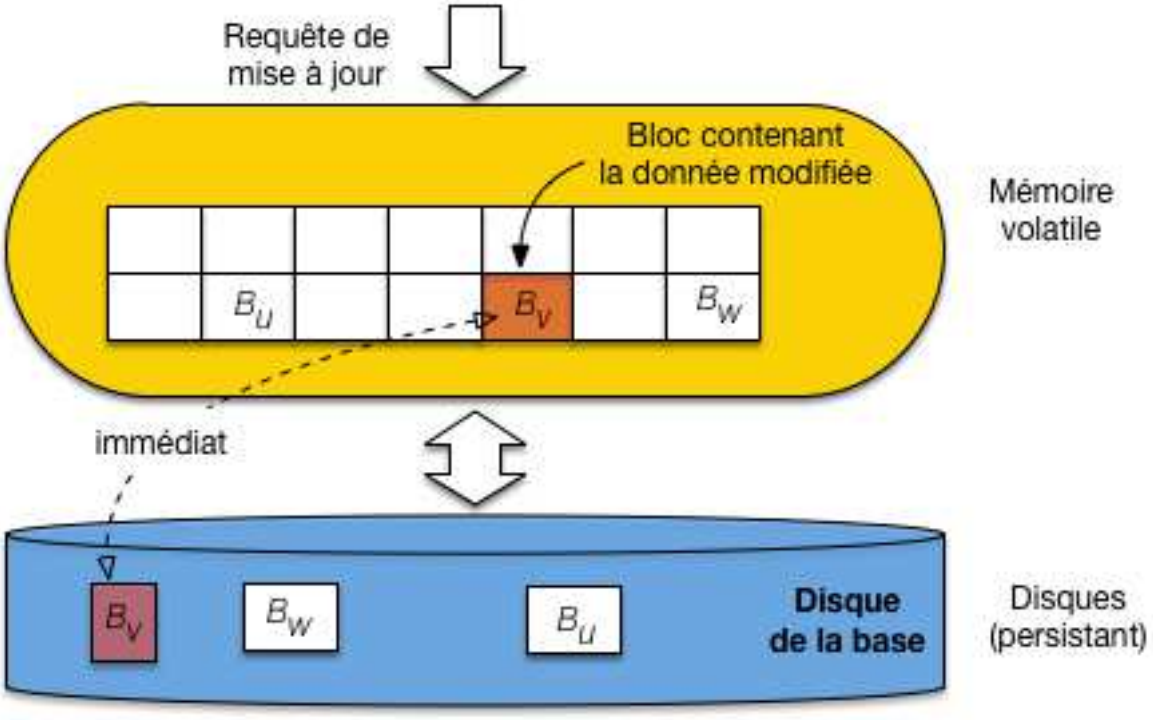
Nous disposons de plusieurs options d'écriture: immédiate, différée, opportuniste.

Si une panne légère survient: le contenu de la RAM est perdu.

Existe-t-il une stratégie possible de reprise sur panne?

Scénario 1: avec écritures opportunistes

Supposons que les écritures fonctionnent en mode opportuniste.



Si un bloc contenant des données non validées est écrit, **l'image avant est écrasée**. Pas de reprise possible.

Vrai, à plus forte raison, avec écritures immédiates.

Scénario 2: avec écritures différées

Tentons une approche plus raisonnée.

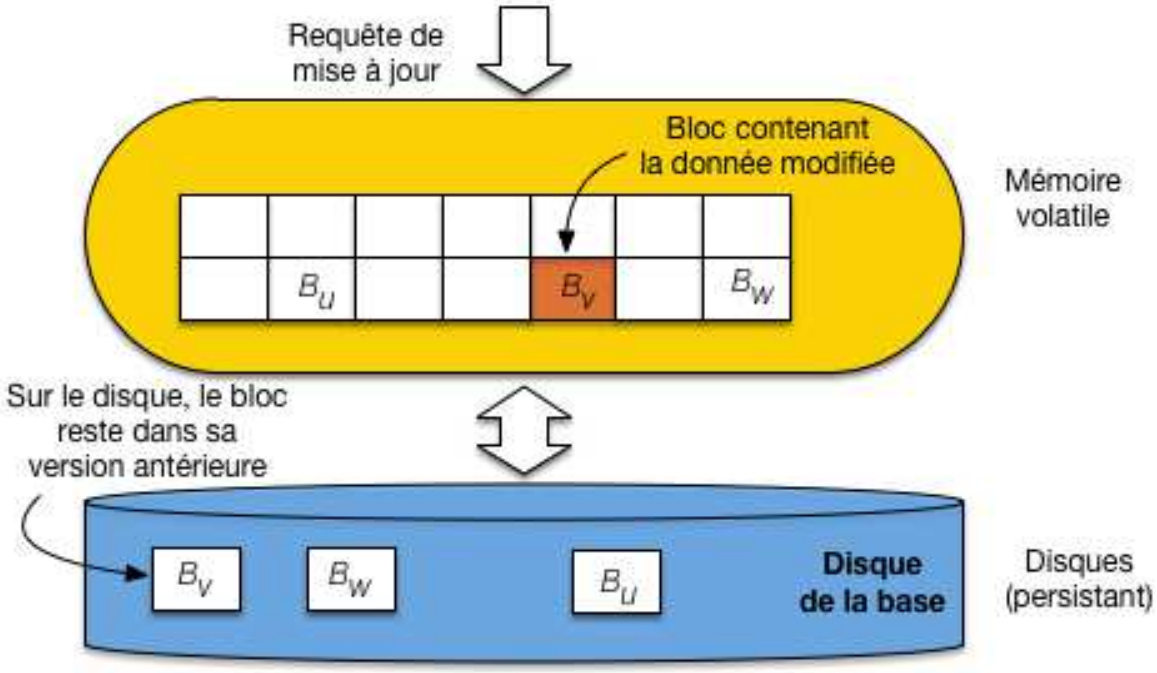
- ne **jamais** écrire un nuplet modifié **avant** le `commit` ;
- au moment du `commit` de T, forcer l'écriture de **tous** les blocs modifiés par T.

Le `commit` et le `rollback`.

- Le `commit` transfère l'image après du buffer vers le disque
- Le `rollback` supprime les blocs modifiés: on en revient à l'image avant.

Pourquoi ça ne marche pas?

Une panne survient **après** quelques écritures mais **avant** l'enregistrement du `commit`.



`rollback` impossible si je ne garde pas **sur disque** les données **après** et **avant** modification.

Résumé: buffer, disque et reprise sur panne

Au moment de l'exécution de l'ordre `commit`

- **l'image avant doit être sur disque**: permet un `rollback` jusqu'à la complétion.
- **l'image après** doit être sur disque: assure la reprise sur panne (légère).

De plus, le mode d'écriture opportuniste doit être privilégié car

- mode différé $\Rightarrow$ encombre le buffer.
- mode immédiat $\Rightarrow$ écritures aléatoires.

La solution s'appelle le **journal des transactions**.

Le journal des transactions (*log*)

Le fichier des transactions, ou *log*, un fichier complémentaire à la base de données.

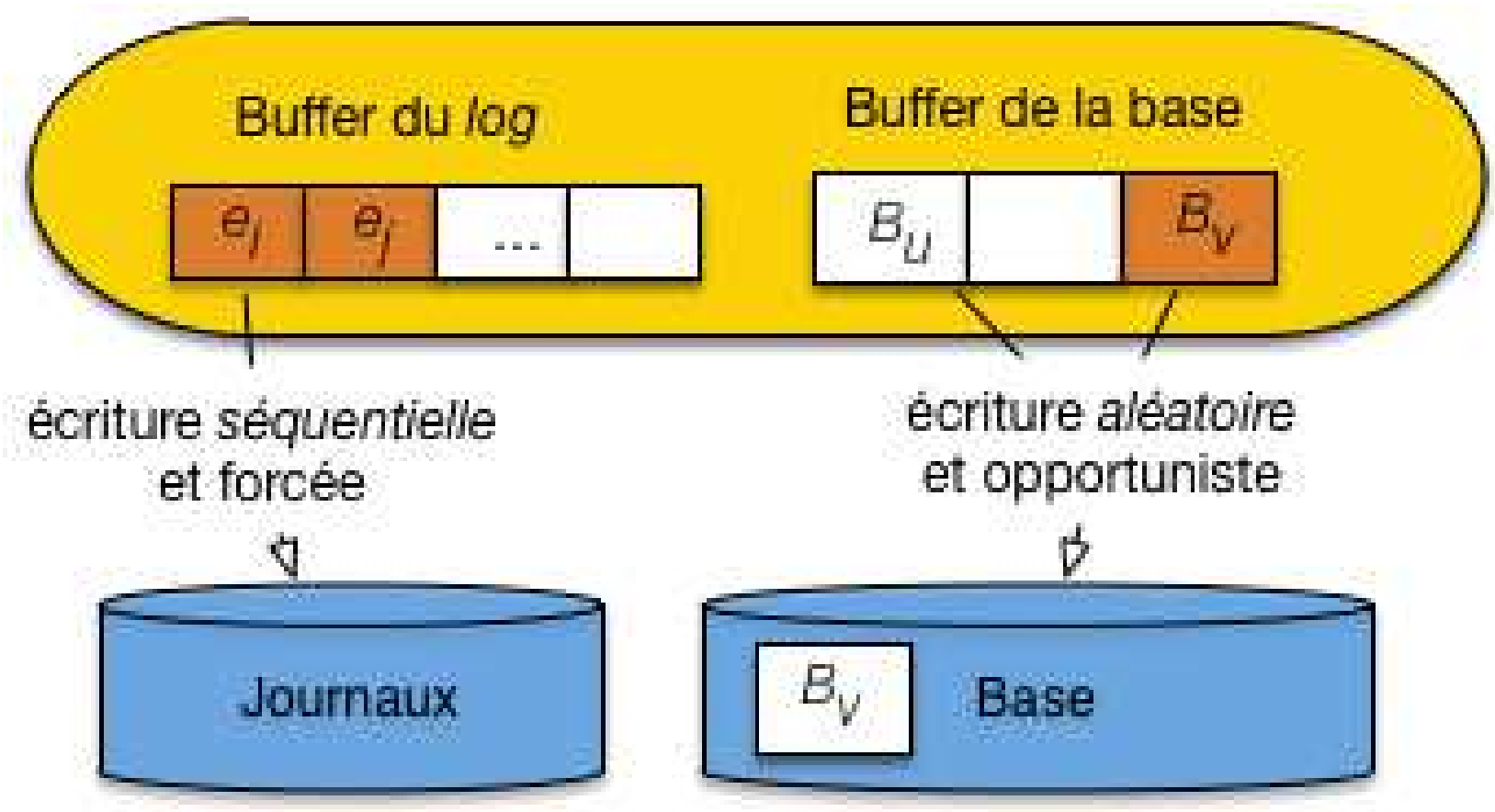
- le système y écrit **séquentiellement**
- le *log* dispose de son propre buffer
- on n'efface **jamais** un enregistrement du *log*

La reprise sur panne repose sur la stratégie suivannte:

- **la base et son buffer** sont en écriture **opportuniste**
- **le *log* et son buffer** sont en écriture **immédiate**

Le *log* contient l'historique des mises à jour.

Le système d'entrées/sorties avec *log*



Contenu du *log*

Toutes les instructions de mise à jour.

Chaque ligne contient un enregistrement d'une des formes suivantes.

- `start(T)`
- `write(T, x, old_val, new_val)`
- `commit(T)`
- `rollback(T)`
- `checkpoint`

Le `write` peut être représenté sous forme **logique** (instructions) ou **physique** (enregistrements).

Le *log* et la gestion des `commit`

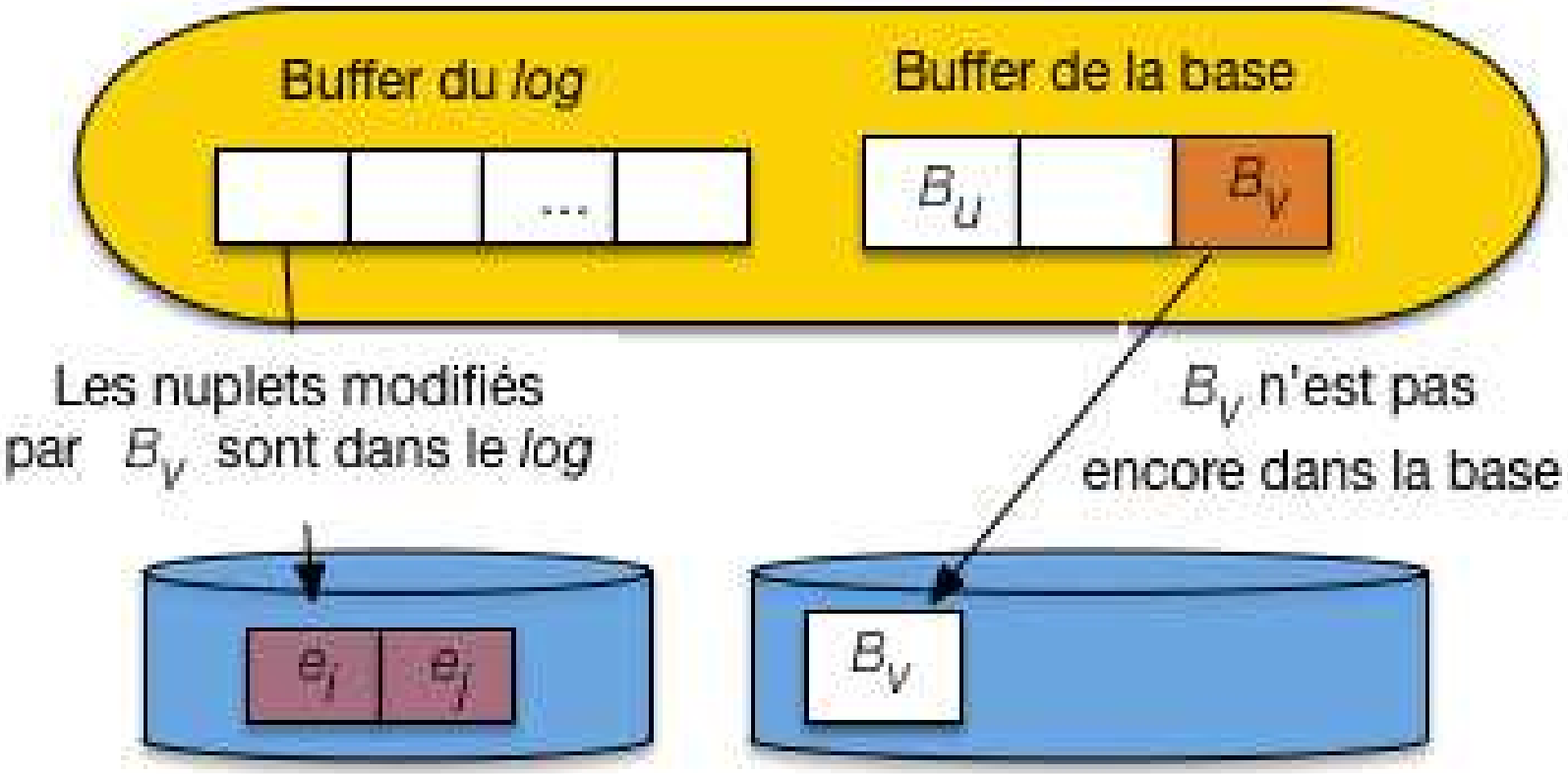
La règle suivante est dite **point de commit**:

- au `commit`, on force l'écriture des modifications effectuées dans le *log*

Conséquences

- **L'état de la base** peut être reconstitué à partir du *log*
- Une transaction est validée quand l'instruction `commit` est écrite dans le *log*

Illustration: après un `commit`



Le *log* et la gestion des `rollback`

Un bloc **modifié** mais pas encore **validé** peut être écrit dans la base.

En cas de panne, il faudra reprendre l'image avant.

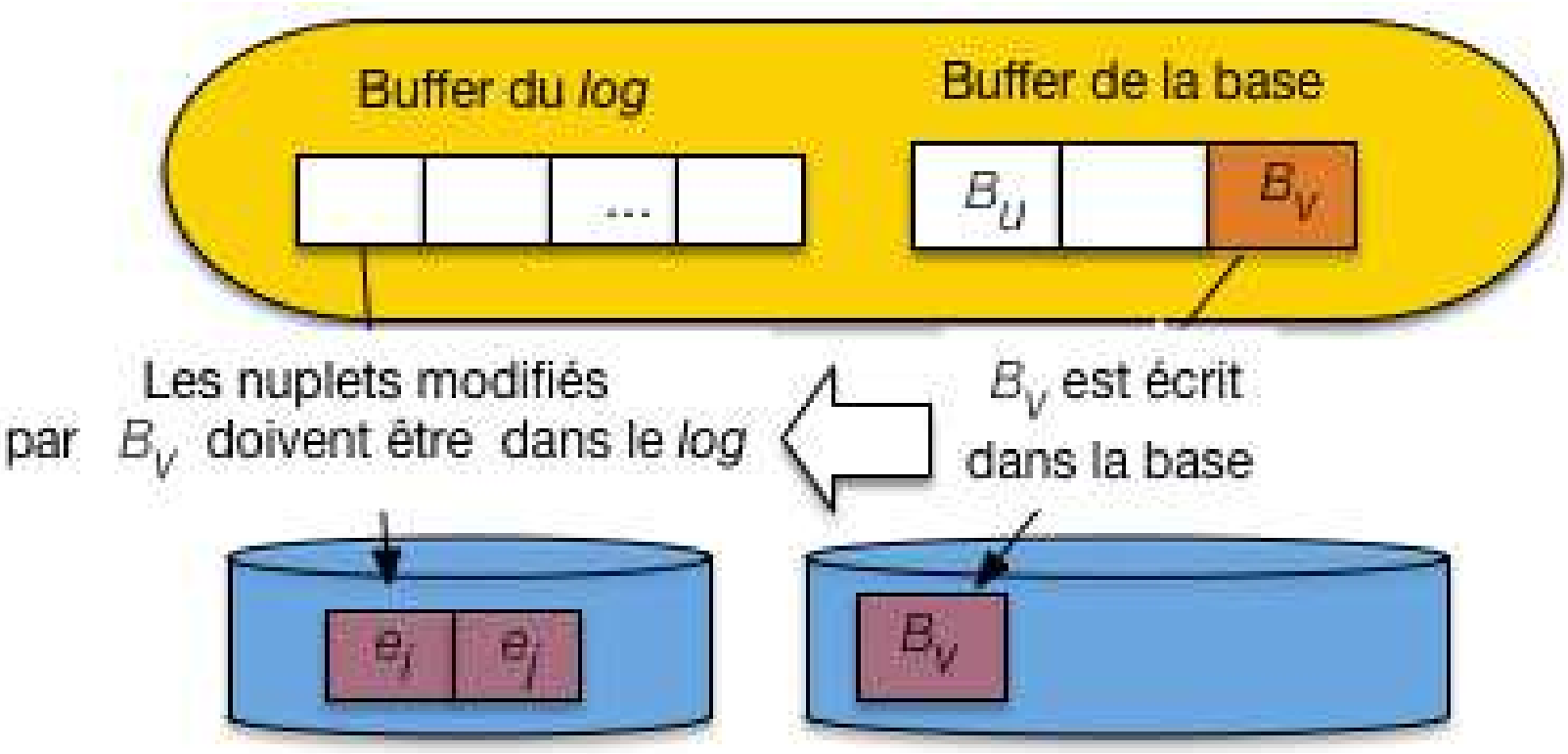
La règle suivante est dite *write-ahead logging*

- quand un bloc modifié est écrit dans la base, il faut **d'abord** écrire les modifications dans le *log*.

Conséquence

- Le *log* contient des modifications qui n'ont pas été validées.

Illustration du *write-ahead*



Résumé: le *log*

Retenir:

- Le *log* est un fichier séquentiel dans lequel on enregistre toutes les mises à jour
- un `commit` est effectif quand l'instruction est écrite dans le *log*
  $\Rightarrow$ l'état de la base est dans le *log*
- un `rollback` peut s'effectuer en reprenant l'image avant dans le *log*
- performances: on laisse la base et son buffer en mode d'écriture opportuniste

Prochaine étape: l'algorithme de reprise sur panne

Refaire et défaire

En cas de panne **légère**, **l'écriture opportuniste** a pour conséquences:

- Des modifications **validées** qui ne sont pas encore dans la base.
- Inversement, des modifications **non validées** qui sont dans la base.

À la reprise, il faut donc

- **Refaire** (REDO) les transactions validées ;
- **Défaire** (UNDO) les transactions en cours.

Ces deux opérations sont basées sur le journal.

Les informations du *log*

On peut reconstituer, à partir du log

- la liste L_V des transactions validées : on trouve un `commit` dans le *log*
- la liste L_A des transactions actives ou annulées : pas de `commit` dans le *log*
- l'image après (`new_val`) et l'image avant (`old_val`) pour chaque transaction.

Ces informations sont nécessaires et suffisantes pour la reprise sur panne

Défaire

Les transactions non validées, L_A, celles qui n'ont pas de `commit` dans le journal

L'algorithme de UNDO prend chaque T de L_A, **dans l'ordre inverse d'exécution**.

Puis, pour chaque `write(T, x, old_val, new_val)`, on écrit `old_val` dans `x`.

Refaire

Les transactions validées, L_V : pour lesquelles on trouve un `commit(T)` dans le *log*

L'algorithme de REDO prend chaque T de L_V, **dans l'ordre d'exécution**.

Puis, pour chaque `write(T, x, old_val, new_val)`, on écrit `new_val` dans `x`.

Et si la reprise subit une panne ?

Il faut recommencer: le REDO est **idempotent**.

L'algorithme de reprise UNDO/REDO

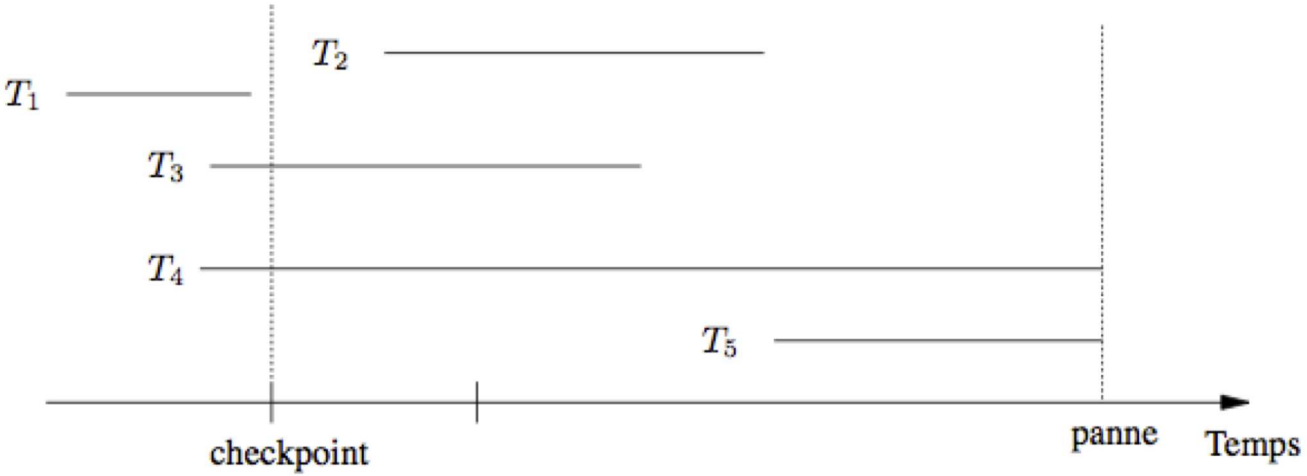
C'est le plus général.

- on constitue la liste des transactions actives L_A et la liste des transactions validées L_V au moment de la panne;
- on **défait** les écritures de L_A
  $\Rightarrow$ attention les annulations se font dans l'ordre inverse de l'exécution initiale;
- on **refait** les écritures de L_V avec le journal.

À propos des *checkpoints*

En cas de panne, il faudrait en principe refaire toutes les transactions du journal, **depuis l'origine de la création de la base**.

Un **checkpoint** écrit sur disque tous les blocs modifiés, ce qui garantit que les données validées par `commit` sont dans la base.



Résumé: reprise sur panne UNDO/REDO

Retenir:

- Le *log* contient la liste des transactions validées; la liste des transactions annulées.
- Une reprise ne prend en compte que celles depuis le dernier *checkpoint*.
- La reprise consiste à parcourir le *log* et à **défaire** les transactions annulées, puis **refaire** les transactions validées.

Schéma général: variantes dans les systèmes, voir exercices.

Panne d'un disque

Panne la plus grave, car on risque de perdre **l'état de la base**.

Solution : **la réplication**. Réfléchissons.

- L'état de la base est dans le fichier journal.
- L'état de la base est **aussi** dans le fichier de la base et le buffer.

Constat : pour qu'il y ait réplication, il est **impératif** de stocker la base et le *log* sur deux disques **distincts**.

Cas 1 : panne du disque de la base

Solution de base:

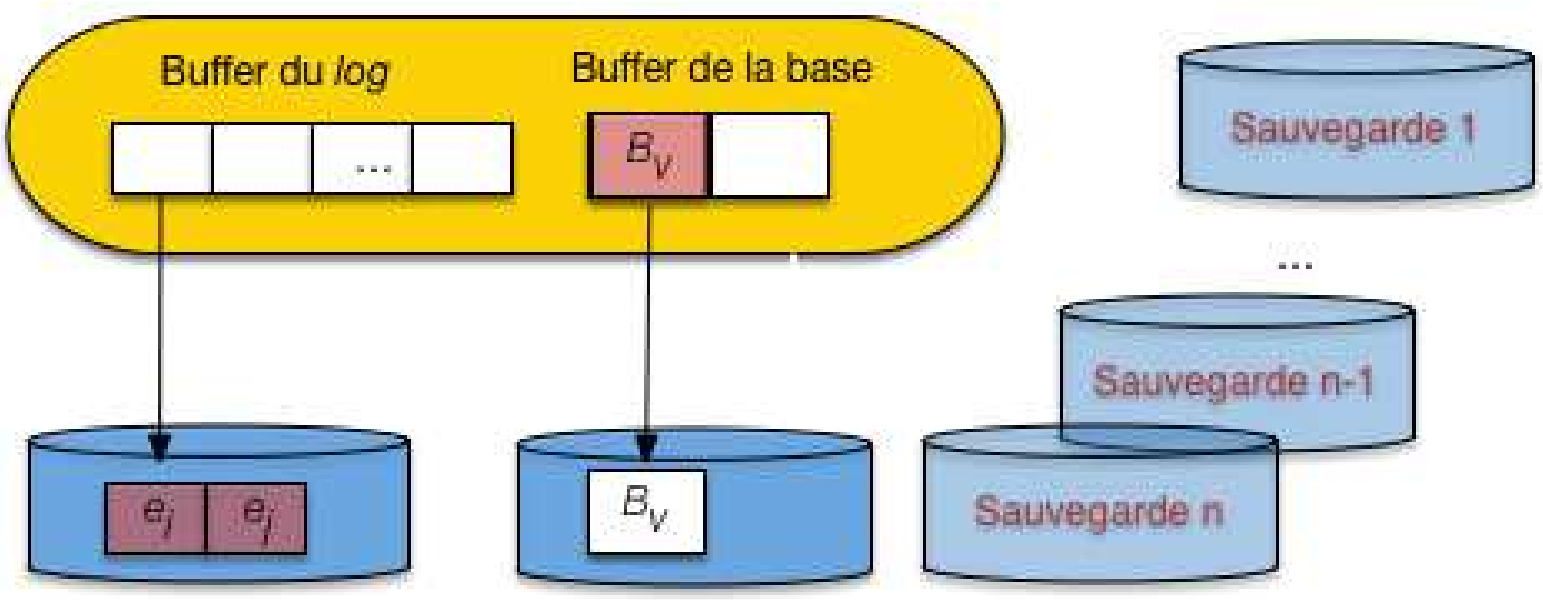
- On réinstalle un disque pour la base.
- On ré-exécute toutes les transactions du *log*.

Inconvénients :

- Phase de latence longue au moment du redémarrage.
- Implique de conserver tous les *log* depuis l'origine.

Panne du disque de la base : avec sauvegarde

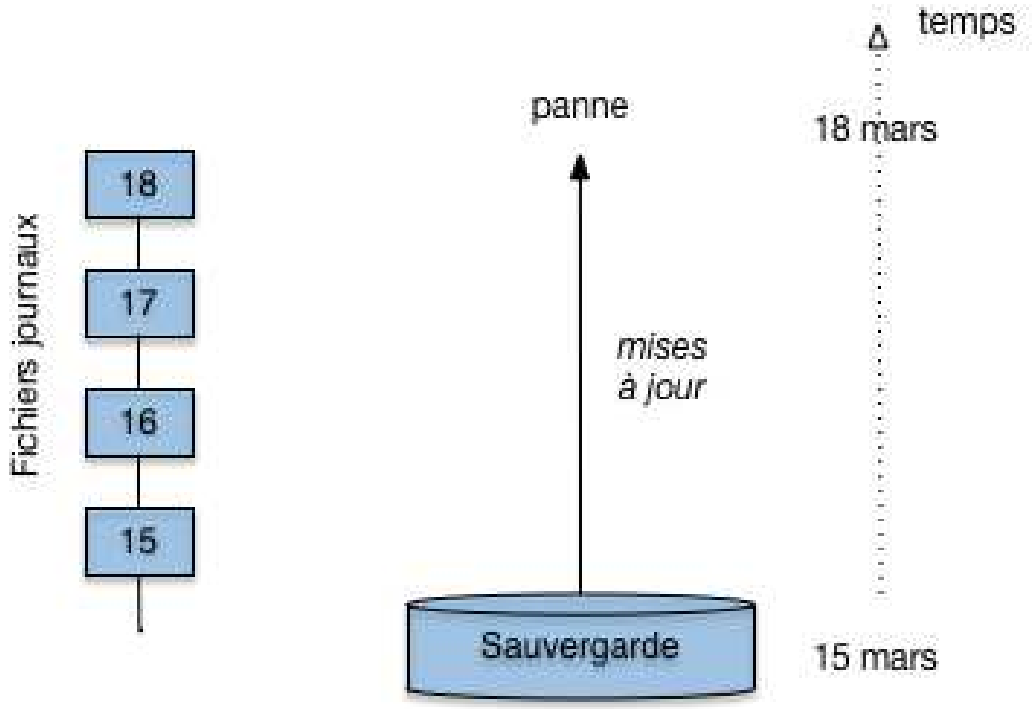
Une **sauvegarde** S_t est une copie de **l'état de la base** à un instant t.



Avec S_t, il suffit de réappliquer le *log* depuis t.

Illustration

Avec une sauvegarde datant du 15 mars, et un fichier *log* par jour.



On applique à la sauvegarde les transactions depuis le 15 mars.

Cas 2 : panne du disque du *log*

Rappel : l'état de la base est dans les fichiers **et** le buffer.

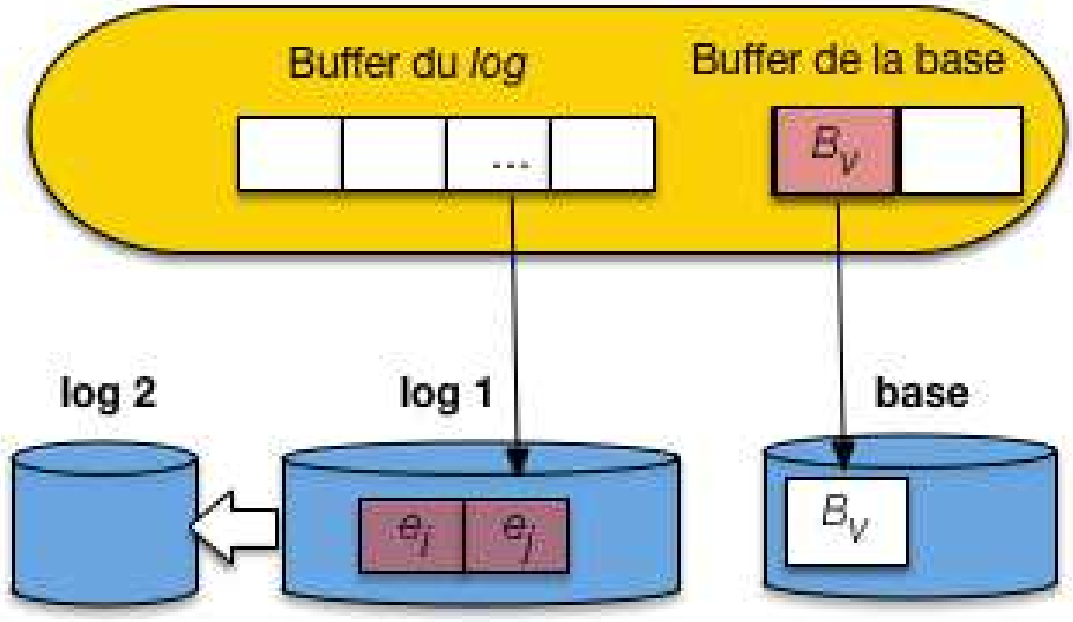
On gère la panne en effectuant un *checkpoint*

- on écrit les blocs modifiés sur le disque de la base,
- on fait les réparations nécessaires et on redémarre.

Assez fragile: *quid* en cas de panne électrique **et** panne du disque de *log* ?

Cas 2, avec réplication du *log*

On peut répliquer le *log* pour plus de sûreté.



Et pourquoi ne pas répliquer la base? On obtient un **système distribué**: c'est pour la semaine prochaine.

Résumé: les pannes de disque

Retenir:

- Le *log* et la base doivent être sur des disques distincts.
- Il faut mettre en place une politique de sauvegarde.
- La reprise combine choix de la sauvegarde et application du *log*.

En principe, on **garantit** la durabilité.

Sûreté totale ?

- Obtenue (asymptotiquement) par **réplication**.
- Une des motivations des systèmes distribués.

Définition du problème

L'état de la base

L'état de la base à un instant t est l'état résultant de l'ensemble des transactions **validées** à l'instant t depuis l'origine.

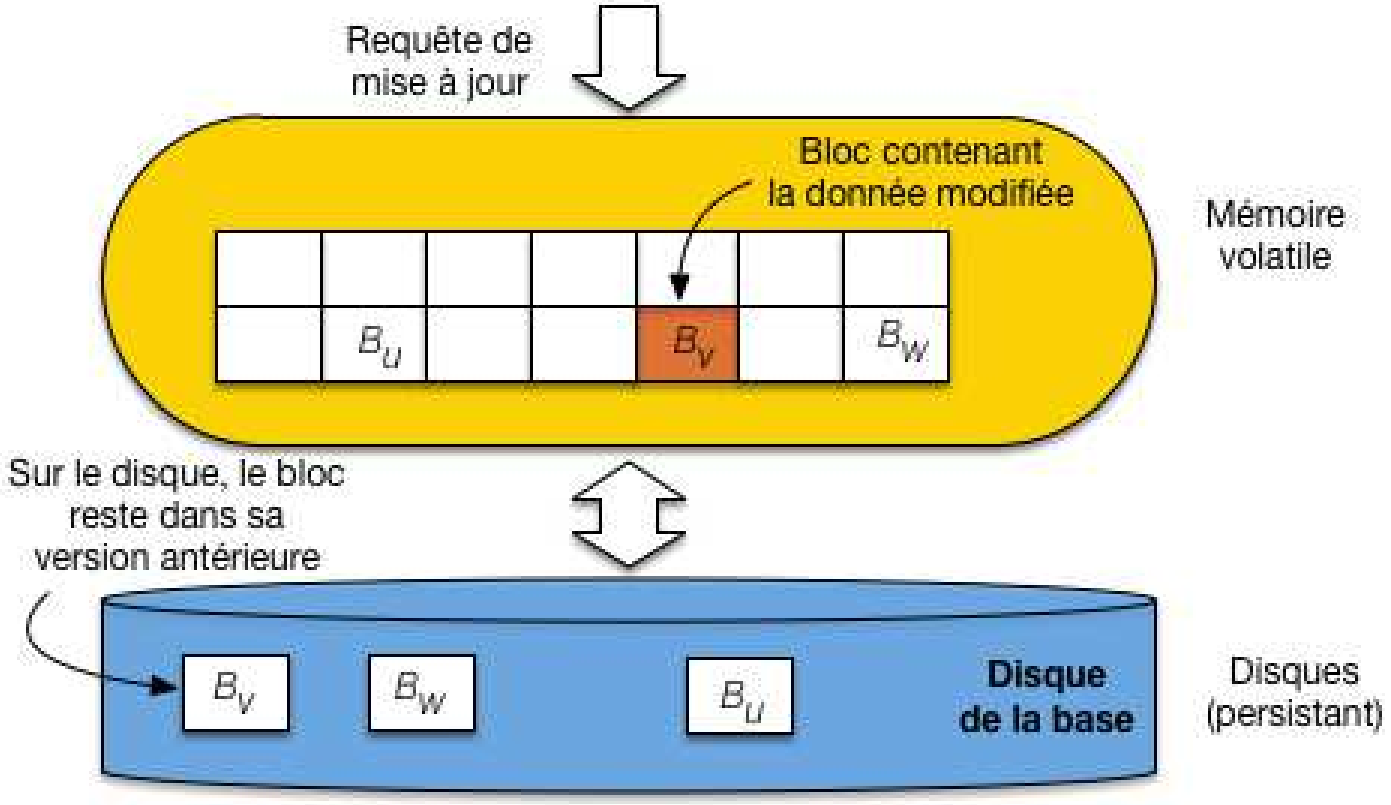
La garantie de durabilité implique qu'en cas de panne, le système récupère l'intégralité de l'état de la base.

Pannes?

- Le SGBD plante.
- le système d'exploitation plante.
- Coupure électrique (le plus fréquent).
- Défaillance d'un disuqe (mais pas des deux!)
- etc.

Où est l'état de la base?

Reprenons l'architecture d'un SGBD avec disque et cache, et une approche simple des écritures.



Règle 1

L'état de la base doit être dans la base

Un premier algorithme

Tentons une approche simpliste pour pouvoir assurer la reprise sur panne.

- ne jamais écrire une donnée modifiée par une transaction T avant que le `commit` n'arrive,
- au moment du `commit` de T, forcer l'écriture de tous les blocs modifiés par T.

Cela suppose qu'on "épingle" (*pin*) en mémoire les blocs modifiés.

Hum, ça ne marche pas...

Pourquoi ça ne marche pas?

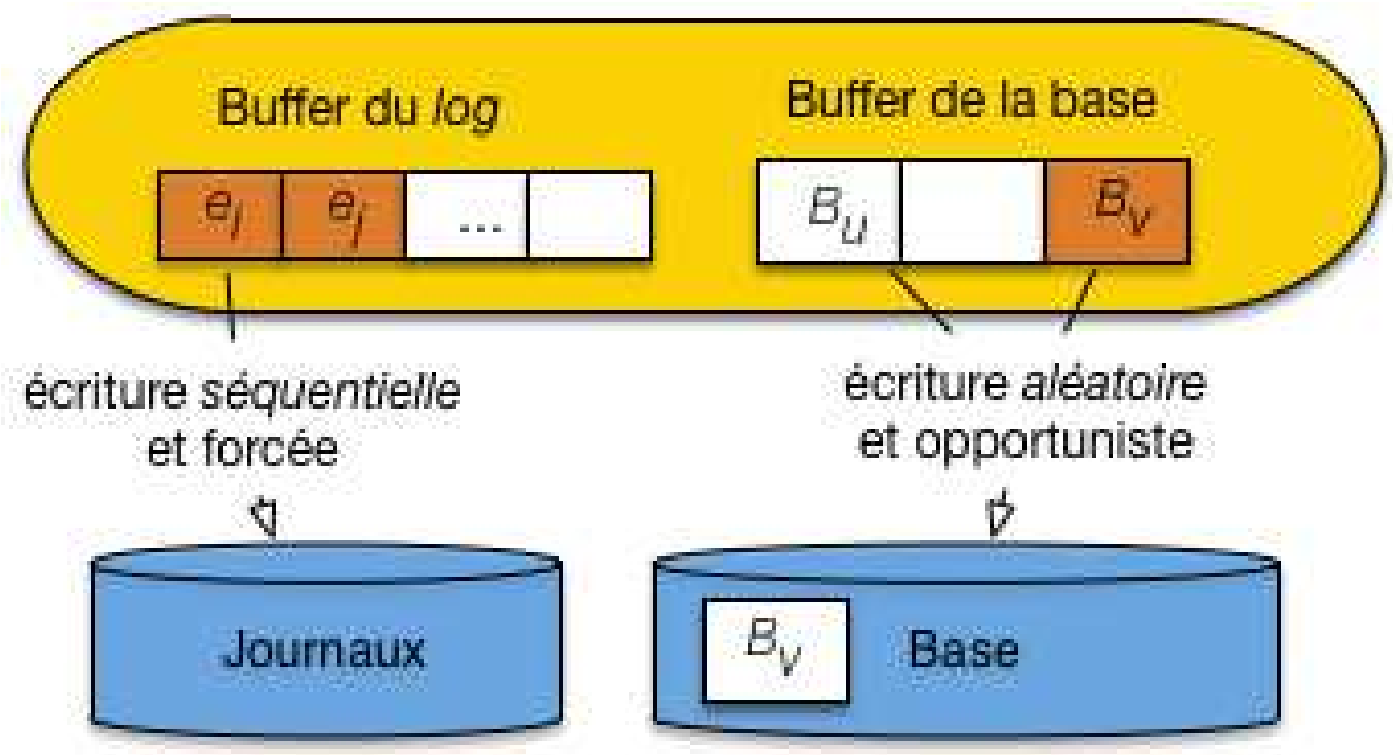
Pour des raisons de performance **et** des raisons de correction.

- **Surcharge du cache.** Le cache peut être trop petit pour stocker toutes les données en cours de modification.
- **Écritures aléatoires.** Au moment du `commit`, on engendre des écritures aléatoires sur le disque: pas bon.
- **Risque sur la recouvrabilité.** Que faire si une panne survient **après** des écritures mais **avant** l'enregistrement du `commit` ?

Le dernier point est important: **J'ai écrasé mon image avant**. je ne peux plus effectuer de `rollback` si je ne garde pas les données **après** et **avant** modification.

Le journal des transactions (*log*)

On règle le(s) problèmes avec un **journal des transactions**.



Règle 1'

L'état de la base doit être sur un disque: soit dans la base, soit dans le journal

Contenu du *log*

Toutes les instructions de mise à jour!

Chaque ligne contient un enregistrement d'une des formes suivantes.

- `start(T)`
- `write(T, x, old_val, new_val)`
- `commit`
- `rollback`
- `checkpoint`

Règle du point de commit

Au moment d'un `commit` le cache du journal doit être écrit sur le disque (écriture forcée). L'état de la base est alors sur le disque.

Refaire et défaire

En cas de panne légère (pas de perte de disque), il faut effectuer deux types d'opérations :

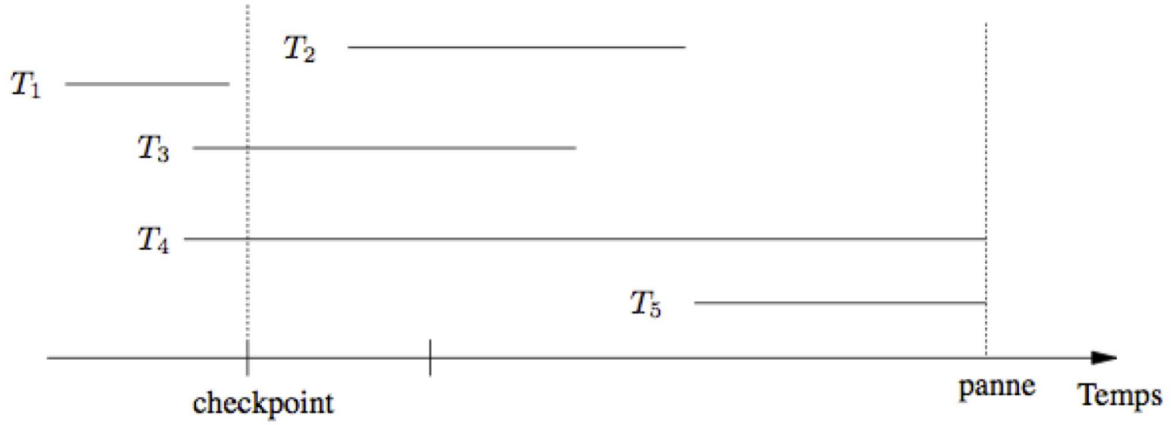
- **Refaire** (REDO) les transactions validées avant la panne ;
  $\Rightarrow$ celles pour lesquelles on trouve un `commit` dans le journal
- **Défaire** (UNDO) les transactions en cours au moment de la panne, qui avaient déjà effectué des mises à jour dans les fichiers de la base.
  $\Rightarrow$ celles qui n'ont ni `commit`, ni `rollback` dans le journal

Ces deux opérations sont basées sur le journal.

À propos des *checkpoints*

En cas de panne, il faudrait en principe refaire toutes les transactions du journal, **depuis l'origine de la création de la base**.

Un **checkpoint** écrit sur disque tous les blocs modifiés, ce qui garantit que les données validées par `commit` sont dans la base.



Un algorithme de reprise

C'est le plus général.

- on constitue la liste des transactions actives L_A et la liste des transactions validées L_V au moment de la panne;
- on annule les écritures de L_A avec le journal
  $\Rightarrow$ attention les annulations se font dans l'ordre inverse de l'exécution initiale;
- on refait les écritures de L_V avec le journal.

On ne force jamais l'écriture des données modifiées (sauf aux *checkpoints*).

Permet d'attendre qu'un *flush* (mise sur disque des blocs modifiées) intervienne naturellement pour qu'elles soient placées sur le disque.

Panne d'un disque

C'est la panne la plus grave.

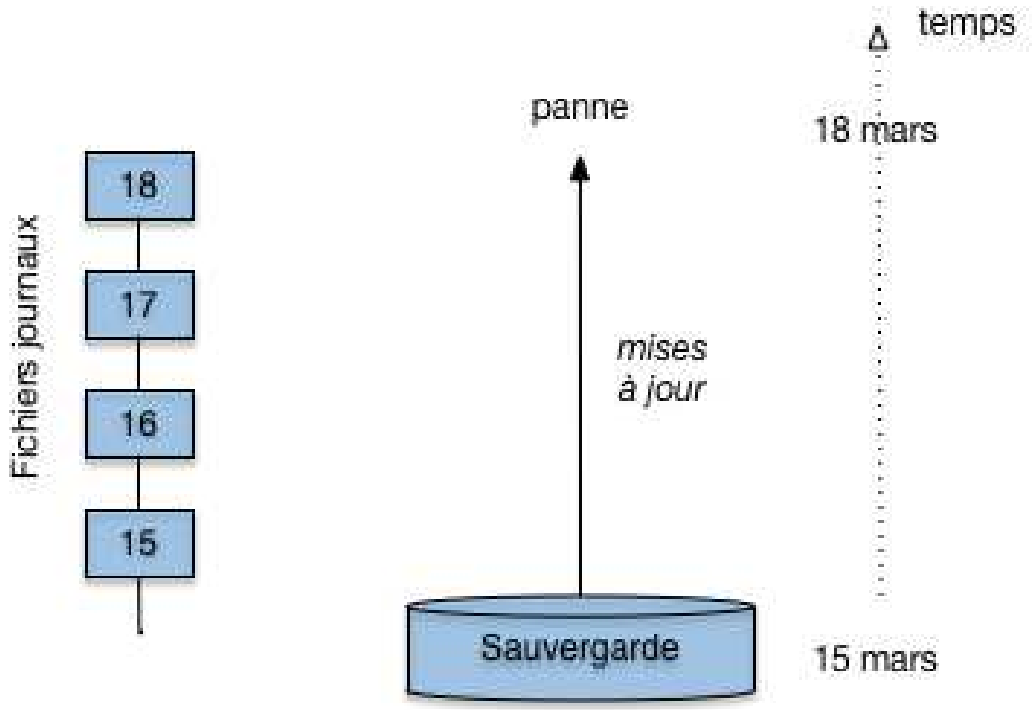
Deux cas.

- **Panne du disque du *log*, D_l.**
  - Assez simple: on écrit les bloc modifiés sur le disque de la base, qui contient alors l'état de la base.
  - On fait les réparations nécessaires sur D_l et on redémarre.
- **Panne du disque de la base, D_b.** Plus délicat.
  - Au pire on fait une reprise depuis l'origine avec les journaux (!)
  - En pratique, on prend la dernière sauvgarde de D_b, et on applique la reprise à la version récupérée.

Ce type de reprise entraîne une phase de latence assez longue au moment du redémarrage.

Illustration

Avec une sauvegarde datant du 15 mars, et des *checkpoints* quotidiens.



L'administrateur réinstalle un disque neuf, y place la sauvegarde du 15 mars, demande ensuite au système une reprise sur panne depuis le 15 mars, à partir du journal.